Server-Factory-Core-Framework

The shared engine behind every Server Factory.

Summary

Core Framework is the Kotlin framework that underpins the Server Factory family of provisioning tools. It provides the common engine and abstractions that projects such as Mail Server Factory build upon, so each “factory” reuses one battle-tested foundation rather than re-implementing provisioning primitives.

Short description

The shared Kotlin framework at the base of the Server Factory ecosystem. It supplies the common provisioning engine, connection abstractions, and installation-step machinery consumed by downstream factories (Mail Server Factory, Web Service Factory, SonarQube Factory, and others).

Long description

Core Framework is the quiet piece of engineering that makes the whole Server-Factory family possible: the reusable engine every individual “factory” product (Mail Server Factory, Web Service Factory, SonarQube Factory, Caching Proxy Factory) is built on top of. The Server Factory approach is declarative — a user describes the infrastructure they want as configuration, and a factory interprets that description to install and initialize software on a target system — and Core Framework is where the machinery common to that pattern actually lives: the connection and transport abstractions that reach every kind of target, the installation-step model that encodes *how* software gets provisioned, and the shared plumbing every factory would otherwise have to write for itself. It is the answer to a structural question every multi-product toolchain eventually faces — where

does the shared engine go? — and getting that answer right once is what keeps the family coherent instead of fragmenting into four subtly-different provisioners. By centralizing this into one Kotlin framework, the family avoids duplicating provisioning logic across products and keeps behavior consistent: a connection type or installation primitive improved in Core Framework benefits every downstream factory. It is almost entirely Kotlin (roughly 990K bytes of Kotlin with a thin Shell layer), reflecting its role as a code library rather than a script collection. Downstream repos link back to it as their canonical dependency (Parallels-Utils, Qemu-Utils, Utils, and the Definitions packs all reference the Core Framework repository as the hub of the ecosystem). Its README is intentionally minimal — it is infrastructure for other projects, versioned via `version.txt`/`version_code.txt` — and it predates the later AI work, making it part of the org's mature DevOps toolchain heritage.

Why we built it

Each provisioning tool needs the same core: ways to connect to targets and steps to install/configure software. Rebuilding that per product would fragment behavior and multiply bugs. Core Framework centralizes it so every factory shares one dependable engine.

Why it's a game-changer

It is the single highest-leverage point in the entire family: a connection type hardened or an installation primitive improved here propagates that correctness and capability to every factory at once, so the whole toolchain compounds off one investment. It is the "build once, reuse everywhere" philosophy applied where it pays the most — the foundation layer of infrastructure automation, where a fix in the right place fixes everything downstream.

What's innovative

- A single reusable provisioning framework abstracting connection + installation-step logic.
- Clean separation between the engine (Core Framework) and product-specific factories.
- Version-pinned distribution (`version.txt`/`version_code.txt`) for reproducible consumption.

Challenges & solutions

- **Avoiding duplicated provisioning logic:** solved by extracting shared machinery into one framework consumed by all factories.
- **Consistent behavior across products:** solved with common abstractions so connection types and steps behave identically everywhere.
- **(UNVERIFIED):** specific internal APIs are not documented in the public README; treat interface details as unverified beyond “shared framework consumed by the factories.”

Tech stack (why + how)

- **Kotlin** — the entire framework (~990K bytes); the language of the Server Factory family.
- **Shell** — minimal supporting scripts.
- **Gradle** — build toolchain (consistent with the family’s `./gradlew` usage).

> Note: GitHub marks the repository as a fork within the Server-Factory org. Not AI-centric; presented as the backbone of the provisioning toolchain.